



BÀI 05: NGOẠI LỆ VÀ XỬ LÝ NGOẠI LỆ

- ❑ Ngoại lệ
- ❑ Xử lý ngoại lệ
- ❑ Phân cấp các lớp ngoại lệ trong Java
- ❑ Khối lệnh try, catch và finally
- ❑ Một số ngoại lệ cơ bản của Java
- ❑ Bắt và xử lý ngoại lệ
- ❑ Tạo ngoại lệ tùy chỉnh



Ngoại lệ

- ❑ Là một sự kiện xảy ra trong quá trình thực thi Chương trình, phá vỡ luồng bình thường của chương trình.
- ❑ Không xử lý ngoại lệ:
 - ❑ Lỗi lan truyền trong chương trình
 - ❑ Chương trình kết thúc bất thường
 - ❑ Tài nguyên không được giải phóng
 - ❑ Chương trình đưa ra kết quả không mong muốn
- ❑ => **bắt và xử lý ngoại lệ**



Ngoại lệ

```
import java.io.*;
import java.util.*;
public class VD_NgoaiLe1 {
    public static void main(String arg[]) {
        int x, fx;
        Scanner inputData = new Scanner(System.in);
        System.out.print("Nhập x: ");
        x = inputData.nextInt();
        fx = 10/x;
        System.out.println ("f(x) = " + fx);
    }
}
```



Nhập x: 0 ↵

Exception in thread "main" [java.lang.ArithmeticException: / by zero](#)
at VD_NgoaiLe1.main([VD_NgoaiLe1.java:10](#))



Ngoại lệ

```
import java.io.*;
import java.util.*;
public class VD_NgoaiLe1 {
    public static void main(String arg[]) {
        int x, fx;
        Scanner inputData = new Scanner(System.in);
        System.out.print("Nhập x: ");
        x = inputData.nextInt();
        fx = 10/x;
        System.out.println ("f(x) = " + fx);
    }
}
```



Nhập x: 1K ↵

Exception in thread "main" [java.util.InputMismatchException](#)
at java.util.Scanner.throwFor(Unknown Source)
at java.util.Scanner.next(Unknown Source)
at java.util.Scanner.nextInt(Unknown Source)
at java.util.Scanner.nextInt(Unknown Source)
at VD_NgoaiLe1.main([VD_NgoaiLe1.java:9](#))



Xử lý ngoại lệ

- ❑ Muốn đảm bảo hệ thống hoạt động tốt trong mọi tình huống và thích ứng với mọi điều kiện thì phải có cơ chế để xử lý được tất cả các *ngoại lệ*, các *tình huống* liên qua tới phạm vi xác định của các phần tử trong các cấu trúc dữ liệu.
- ❑ Một ngoại lệ (exception) trong chương trình Java: Là dấu hiệu chỉ ra rằng có sự xuất hiện một điều kiện không bình thường nào đó.
- ❑ Ví dụ: chương trình yêu cầu dữ liệu từ một tệp mà tệp đó lại không có trên đĩa, hay chỉ số của phần tử mảng vượt ra ngoài phạm vi xác định, v.v.
- ❑ Java cung cấp một cơ chế để kiểm tra một cách có hệ thống các điều kiện ngoại lệ. Để xử lý ngoại lệ trong Java, chúng ta sử dụng khối lệnh *try-catch-finally*.



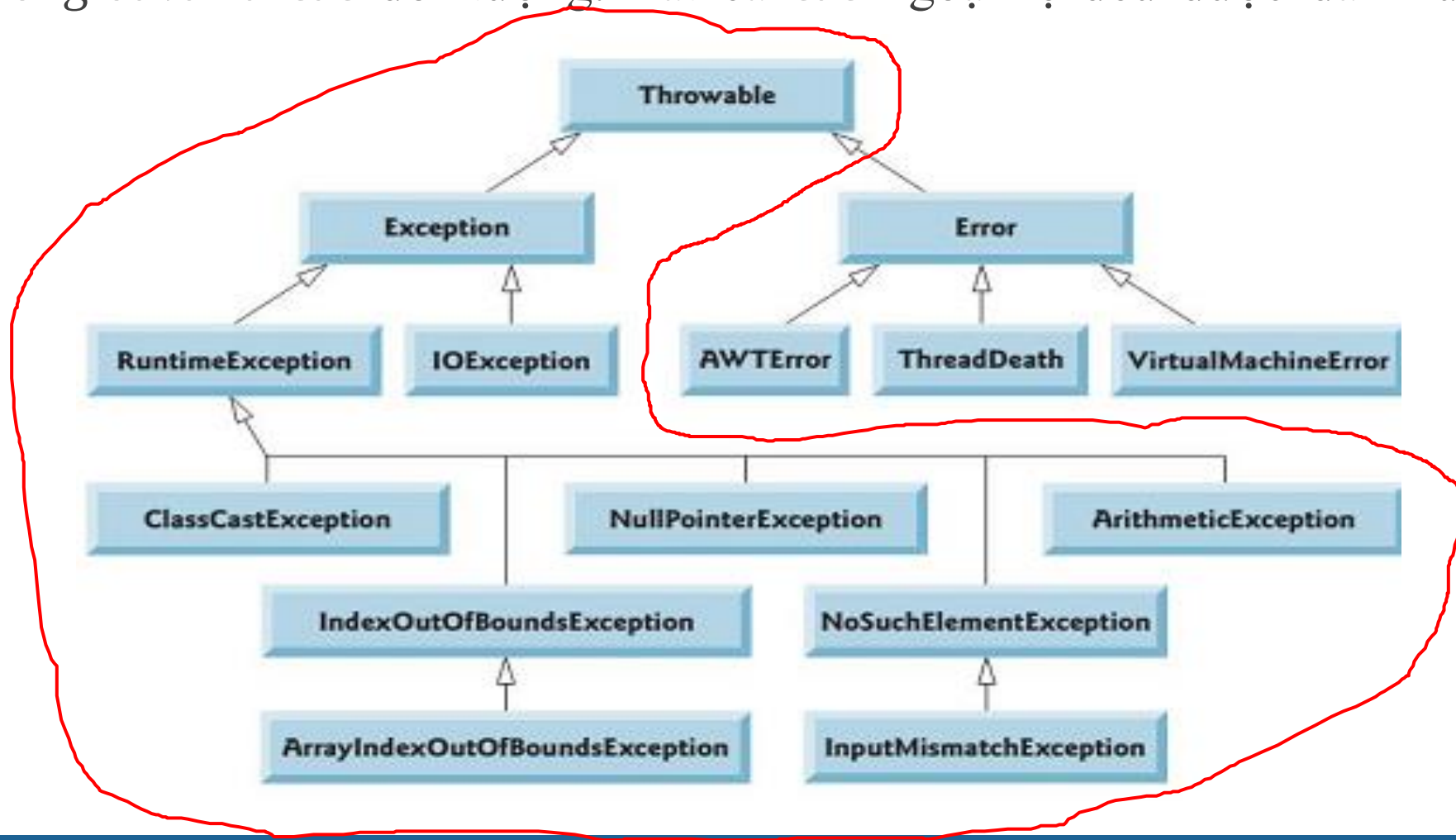
Xử lý ngoại lệ

- ☐ Đóng gói các thông tin và phương thức của ngoại lệ trong các lớp
- ☐ Tất cả các ngoại lệ xuất hiện được coi như là một đối tượng của các lớp kế thừa từ lớp Throwable hoặc các lớp con của nó
- ☐ **Throwable có 2 lớp con trực tiếp:**
 - ☐ **Error**: Các ngoại lệ nghiêm trọng, chương trình không thể quản lý
 - ☐ **Exception**: Các ngoại lệ có thể kiểm soát
- ☐ **Ngoại lệ che được và ngoại lệ không che được:**
 - ☐ Ngoại lệ che được (Unchecked exceptions): Không bắt buộc phải xử lý trong chương trình. Gồm mọi ngoại lệ thuộc RuntimeException và các lớp con
 - ☐ Ngoại lệ không che được (Checked Exceptions): Bắt buộc phải xử lý trong chương trình.



Phân cấp các lớp Ngoại lệ trong Java

Ngoại lệ trong Java là các đối tượng. Tất cả các ngoại lệ đều được dẫn xuất từ lớp *Throwable*





Khối lệnh try, catch và finally

```
try {
```

```
// Khối try
```

```
<Các câu lệnh có khả năng gây ra ngoại lệ>
```

```
}
```

```
catch (<Kiểu ngoại lệ 1> <Tham biến 1>) {
```

```
// Khối catch
```

```
<Các câu lệnh xử lý khi xuất hiện kiểu ngoại lệ 1>
```

```
}
```

```
...
```

```
catch (<Kiểu ngoại lệ n> <Tham biến n>) {
```

```
// Khối catch
```

```
<Các câu lệnh xử lý khi xuất hiện kiểu ngoại lệ n>
```

```
}
```

```
finally {
```

```
// Khối finally
```

```
< Các câu lệnh luôn được thực hiện >
```

```
}
```




Khối lệnh try, catch và finally

❑ Khối try/catch

- ❑ Đặt đoạn mã có khả năng xảy ra ngoại lệ trong khối **try**
- ❑ Đặt đoạn mã xử lý ngoại lệ trong khối **catch**
- ❑ Khi xảy ra ngoại lệ trong khối **try**, các câu lệnh trong khối **catch** sẽ được thực hiện tùy vào kiểu của ngoại lệ.
- ❑ Sau khi thực hiện xong khối catch, điều khiển sẽ được trả lại cho chương trình.

❑ Khối finally:

- ❑ Khi một ngoại lệ xảy ra, chương trình dừng lại, một số công việc “dọn dẹp” có thể sẽ không được thực hiện (ví dụ như đóng file).
- ❑ Khối **finally** đảm bảo rằng các câu lệnh trong đó luôn được thực hiện, kể cả khi ngoại lệ xảy ra.



Khởi lệnh try, catch và finally

```
import java.util.*;
public class VD_NgoaiLe1 {
    public static void main(String arg[]) {
        int x, fx;
        Scanner inputData = new Scanner(System.in);
        System.out.print("Nhập x: ");
        try {
            x = inputData.nextInt();
            fx = 10/x;
        } catch (Exception e) {
            System.out.println ("Co loi xay ra");
            fx=0;
        }
        finally {
            System.out.println ("luôn thực hiện");
        }
        System.out.println ("f(x) = " + fx);
    }
}
```



Khởi lệnh try, catch và finally

```
import java.util.*;
public class VD_NgoaiLe1 {
    public static void main(String arg[]) {
        int x, fx;
        Scanner inputData = new Scanner(System.in);
        System.out.print("Nhập x: ");
        try {
            x = inputData.nextInt();
            fx = 10/x;
        } catch (InputMismatchException e) {
            System.out.println ("Loi nhap sai dinh dang "+e.getMessage());
            fx=0;
        } catch (ArithmeticException e) {
            System.out.println ("Loi chia cho 0 "+e.getMessage());
            fx=1;
        }
        finally {
            System.out.println ("Luon thuc hien");
        }
        System.out.println ("f(x) = " + fx);
    }
}
```



Khởi lệnh try, catch và finally

TÓM LẠI: try-catch-finally

- ❑ Các ngoại lệ xảy ra khi gặp lỗi.
- ❑ Có thể bắt và xử lý các ngoại lệ bằng cách sử dụng khối try/catch. Nếu không chương trình sẽ kết thúc ngay (với ứng dụng console) hoặc tiếp tục tồn tại (với ứng dụng GUI).
- ❑ Khi bắt ngoại lệ, phải biết rõ kiểu ngoại lệ cần bắt. Có thể dùng kiểu cha Exception.
- ❑ Để chắc chắn việc “dọn dẹp” luôn được thực hiện, dùng khối finally. Có thể kết hợp try/catch/finally.



Một số ngoại lệ cơ bản của Java

- ☐ `ArrayIndexOutOfBoundsException`: Chỉ số phần tử mảng vượt khỏi phạm vi xác định của mảng,
- ☐ `NullPointerException`: Biến tham chiếu Đ.Tượng chưa được khởi tạo
- ☐ `ClassCastException`: Lỗi ép kiểu
- ☐ `IllegalArgumentException`: Lỗi liên quan đến tham biến
- ☐ `ArithmeticException`: Lỗi số học, như chia cho 0,
- ☐ `NumberFormatException`: Lỗi định dạng số học
- ☐ `IOException`: Lỗi vào/ra,
- ☐ `FileNotFoundException`: File không tìm thấy (hoặc không tồn tại)
- ☐ `EOFException`: Lỗi truy cập tệp (đã ở cuối tệp)
- ☐ `AWTException`: Ngoại lệ liên quan đến GUI
- ☐ Ngoài ra: `LinkError`, `ThreadDeath`, `VirtualMachineError`



Bắt và xử lý ngoại lệ

Khi ngoại lệ xảy ra, có 3 cách thức xử lý:

- ☐ Bỏ qua (chỉ áp dụng với ngoại lệ che được - unchecked exceptions)
- ☐ Xử lý tại nơi xảy ra ngoại lệ: Sử dụng khối lệnh try ... catch ... finally
- ☐ Xử lý ở vị trí khác, thời điểm khác trong chương trình => **ủy nhiệm ngoại lệ**



Ủy nhiệm ngoại lệ

- ❑ Ngoại lệ có thể không cần xử lý tại chỗ mà ủy nhiệm cho mức cao hơn xử lý
- ❑ Ở mức nào có phần xử lý ngoại lệ thì ngoại lệ được xử lý tại mức đó
- ❑ Mức cao nhất có thể được ủy nhiệm là máy ảo JVM
- ❑ Cú pháp:
[<Phạm vi>] [<Kiểm soát truy nhập>] <Tên phương thức> **throws**
ExceptionType {<Thân phương thức>};
- ❑ Các ngoại lệ RuntimeException mặc định được ủy nhiệm cho các mức cao hơn



Ủy nhiệm ngoại lệ

```
public class VD_NgoaiLe2 {  
    public static void main(String[] args) {  
        try {  
            divide(10, 0);  
        } catch (ArithmeticException e) {  
            System.out.println("Không thể chia cho số 0!");  
        }  
    }  
  
    public static void divide(int a, int b) throws ArithmeticException {  
        int result = a / b;  
        System.out.println("Kết quả là: " + result);  
    }  
}
```




Lợi ích khi ủy nhiệm ngoại lệ

- ☐ Dễ sử dụng:
 - ☐ Chương trình dễ đọc và an toàn
 - ☐ Chuyển ngoại lệ đến vị trí có khả năng xử lý
- ☐ Tách xử lý ngoại lệ khỏi những đoạn mã thông thường
- ☐ Không bỏ sót ngoại lệ
- ☐ Gom nhóm và phân loại ngoại lệ



Tung ngoại lệ cưỡng bức

- ❑ Phương thức được gọi (called method) có thể tung ngoại lệ cưỡng bức và ủy nhiệm xử lý ngoại lệ đó cho phương thức gọi nó (calling method)
- ❑ Thường sử dụng khi kiểm soát dữ liệu vào/ra hoặc một bước nào đó trong quá trình xử lý dữ liệu
- ❑ Cú pháp: Sử dụng từ khóa throw:
[<Phạm vi>] [<Kiểm soát truy nhập>] <Tên phương thức> **throws** ExceptionType {
 <Thân phương thức>;
 throw new ExceptionTypeConstruct();
};
- ❑ Nếu ExceptionType thuộc RuntimeException thì không cần sử dụng throws để ủy nhiệm



Tung ngoại lệ cưỡng bức

```
public class VD_NgoaiLe2 {  
    public static void main(String[] args) {  
        try {  
            checkAge(15);  
        } catch (IllegalArgumentException e) {  
            System.out.println("Ngoại lệ xảy ra: " + e.getMessage());  
        }  
    }  
  
    public static void checkAge(int age) {  
        if (age < 18) {  
            throw new IllegalArgumentException("Tuổi phải từ 18 trở lên để đăng ký.");  
        } else {  
            System.out.println("Đăng ký thành công!");  
        }  
    }  
}
```



Tạo ngoại lệ tùy chỉnh

- ❑ Chúng ta có thể tự tạo lớp ngoại lệ để phục vụ các mục đích riêng.
- ❑ Lớp ngoại lệ mới phải kế thừa từ lớp Exception hoặc lớp dẫn xuất của lớp này.
- ❑ Có thể cung cấp hai constructor (toán tử tạo lập)
 - ❑ Constructor mặc định (không tham số)
 - ❑ Constructor nhận một tham số String và truyền tham số này cho phương thức khởi tạo của lớp cha



Tạo ngoại lệ tùy chỉnh

Định nghĩa lớp ngoại lệ

//file TuoiKhongHopLeException.java

```
public class TuoiKhongHopLeException extends Exception {  
    public TuoiKhongHopLeException(String message) {  
        super(message);  
        System.out.println("Ngoại lệ tự tạo!");  
    }  
}
```



Tạo ngoại lệ tùy chỉnh

Sử dụng ngoại lệ

//file VD_NgoaiLe2.java

```
public class VD_NgoaiLe2 {  
    public static void main(String[] args) {  
        try {  
            checkAge(15);  
        } catch (TuoiKhongHopLeException e) {  
            System.out.println("Ngoại lệ xảy ra: " + e.getMessage());  
        }  
    }  
    public static void checkAge(int age) throws TuoiKhongHopLeException{  
        if (age < 18) {  
            throw new TuoiKhongHopLeException("Tuổi phải từ 18 trở lên để đăng ký.");  
        } else {  
            System.out.println("Đăng ký thành công!");  
        }  
    }  
}
```



BÀI TẬP

- ❑ Bài 1: Sử dụng cấu trúc try, và 2 khối catch xây dựng chương trình nhập vào 2 số nguyên a, b và in ra kết quả a/b. Yêu cầu bắt các ngoại lệ khi nhập a, b và khi thực hiện phép a/b.
- ❑ Bài 2: Dùng mệnh đề Throw để tạo ra ngoại lệ khi nhập tuổi cho nhân viên (không âm và không lớn hơn 62 tuổi)
- ❑ Bài 3: Xây dựng hàm nhậpTuoi() và dùng mệnh đề Throws để chuyển ngoại lệ ra ngoài hàm và thực hiện bắt và xử lý các ngoại lệ khi gọi hàm nhậpTuoi().



BÀI 05: NGOẠI LỆ VÀ XỬ LÝ NGOẠI LỆ

- ❑ Ngoại lệ
- ❑ Xử lý ngoại lệ
- ❑ Phân cấp các lớp ngoại lệ trong Java
- ❑ Khối lệnh try, catch và finally
- ❑ Một số ngoại lệ cơ bản của Java
- ❑ Bắt và xử lý ngoại lệ
- ❑ Tạo ngoại lệ tùy chỉnh



Sinh viên tự học

- ☐ Làm bài tập trắc nghiệm trên hệ thống LMS
- ☐ Bài tập kỹ năng (nếu có)



TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

Thank you!

THÁI NGUYÊN, THÁNG NĂM